

CSCE 548 – Project 4 System Deployment and Testing Report Student:

Ayden Mathews

Course: CSCE 548

Project: Layered Architecture CRUD System

1. Project Overview

This project implements a layered software architecture consisting of four primary layers: data layer, business layer, service layer, and client layer. The goal of the project is to demonstrate how a multi tier architecture allows different parts of an application to work together while remaining modular and maintainable. The system manages a simple student database and supports full CRUD operations (Create, Read, Update, Delete). The service layer exposes REST endpoints using Flask, which interact with the business logic and data access layers. A console-based client program is used to test and demonstrate the system functionality.

2. Technologies Used

Technology	Purpose
Python	Primary programming language
Flask	REST API service layer
SQLite	Database storage
Requests Library	Client HTTP requests
VS Code	Development environment

3. Repository Setup

To run the project, download the repository from GitHub and open the project folder (csce548_project2) in an IDE such as VS Code. The project contains the following files: • data_layer.py – Handles all database operations • business_layer.py – Implements business logic and validation • service.py Flask REST API service • client.py – Console-based client used for testing API endpoints • project2.db– SQLite database storing student records • requirements.txt – Project dependencies

4. Running the Backend Service

Activate the Python virtual environment and run the Flask service: 1. Activate virtual environment: `.\venv\Scripts\activate` 2. Start the service: `python service.py` The backend server will start locally at: `http://127.0.0.1:5000` This service exposes REST endpoints that allow the client application to perform CRUD operations.

5. Client Application Testing

The client application is used to test the API endpoints. The client communicates with the Flask service using HTTP requests through the Python requests library. Run the client using: `python client.py` The client performs the following sequence of operations: • Create a new student record • Retrieve the student by ID • Update the student's information • Retrieve the updated student • Delete the student • Confirm the student record was removed These steps verify that the full CRUD workflow operates correctly.

6. Database Verification

The SQLite database file (project2.db) stores all student records. Database verification was performed using DB Browser for SQLite. During testing, the database table named 'student' displayed the following fields: • id • name • email After running the client operations, the database showed the updated student record, confirming that the data layer successfully processed the requests.

7. System Architecture Flow

The system follows a layered architecture model: Client Layer → Service Layer → Business Layer → Data Layer → Database Each layer performs a specific role in the application: • Client Layer sends HTTP requests. • Service Layer exposes REST endpoints. • Business Layer processes logic and validation. • Data Layer executes database operations. • Database persists the data. This design ensures modularity and separation of concerns.

8. Final Result

The system successfully demonstrated full CRUD functionality using a layered architecture. Testing confirmed that the client, service, business, and data layers communicate correctly, and the database updates reflect the operations performed by the application. This confirms the successful completion of the project requirements